

AMS 580 Quiz 5

Kachun Huang

2025-04-23

AMS 580 Quiz 5

Import Library

```
library(factoextra)
library(stats)
library(cluster)
```

```
## Warning: package 'cluster' was built under R version 4.3.3
```

```
library(ggplot2)
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
library(ggbiplot)
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
library(kernlab)
```

```
## Warning: package 'kernlab' was built under R version 4.3.3
```

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.3.3
```

Import Data

```
data <- read.csv("/Users/thomashuang/Downloads/GreatUnknown.csv")
str(data)
```

```
## 'data.frame':    4601 obs. of  13 variables:
## $ w1 : num  0.32 0.14 1.23 0.63 0.63 1.85 1.92 1.88 0.61 0.19 ...
## $ w2 : num  0 0.21 0.19 0.31 0.31 0 0 0 0.3 0.38 ...
## $ w3 : num  0 0.94 0.25 0.63 0.63 0 0.64 0 0.76 0 ...
## $ w4 : num  0.32 0.14 0.06 0.31 0.31 0 0.96 0 0 0 ...
## $ w5 : num  0 0 0.32 0 0 0 0 0 3.53 0.06 ...
## $ w6 : num  0 0 0 0 0 0 0 0 0 0 ...
## $ w7 : num  0 0 0.06 0 0 0 0 0 0 0 ...
## $ w8 : num  0 0 0.06 0 0 0 0 0 0 0 ...
## $ w9 : num  0 0 0.01 0 0 0 0 0 0 0.04 ...
## $ w10: num  0 0.132 0.143 0.137 0.135 0.223 0.054 0.206 0.271 0.03 ...
## $ w11: num  0 0 0 0 0 0 0 0 0 0 ...
## $ w12: num  0 0.18 0.184 0 0 0 0.054 0 0.203 0.081 ...
## $ y  : int  1 1 1 1 1 1 1 1 1 1 ...
```

Data Pre-processing

scale

```
data <- na.omit(data)
y <- factor(data$y)
data <- scale(subset(data,select=-c(y)))
data <- as.data.frame(cbind(data,y))
str(data)
```

```
## 'data.frame':    4601 obs. of  13 variables:
## $ w1 : num  0.0116 -0.2561 1.3647 0.4725 0.4725 ...
## $ w2 : num  -0.292 0.245 0.194 0.5 0.5 ...
## $ w3 : num  -0.3713 1.0866 0.0164 0.6058 0.6058 ...
## $ w4 : num  0.0862 -0.1318 -0.2287 0.0741 0.0741 ...
## $ w5 : num  -0.168 -0.168 0.46 -0.168 -0.168 ...
## $ w6 : num  -0.329 -0.329 -0.329 -0.329 -0.329 ...
## $ w7 : num  -0.1853 -0.1853 -0.0138 -0.1853 -0.1853 ...
## $ w8 : num  -0.197 -0.197 -0.132 -0.197 -0.197 ...
## $ w9 : num  -0.158 -0.158 -0.117 -0.158 -0.158 ...
## $ w10: num  -0.51425 -0.026 0.01468 -0.00751 -0.01491 ...
## $ w11: num  -0.155 -0.155 -0.155 -0.155 -0.155 ...
## $ w12: num  -0.308 0.424 0.44 -0.308 -0.308 ...
## $ y  : num  2 2 2 2 2 2 2 2 2 2 ...
```

Factorize Response

```
data$y <- factor(data$y)
str(data)
```

```
## 'data.frame': 4601 obs. of 13 variables:
## $ w1 : num 0.0116 -0.2561 1.3647 0.4725 0.4725 ...
## $ w2 : num -0.292 0.245 0.194 0.5 0.5 ...
## $ w3 : num -0.3713 1.0866 0.0164 0.6058 0.6058 ...
## $ w4 : num 0.0862 -0.1318 -0.2287 0.0741 0.0741 ...
## $ w5 : num -0.168 -0.168 0.46 -0.168 -0.168 ...
## $ w6 : num -0.329 -0.329 -0.329 -0.329 -0.329 ...
## $ w7 : num -0.1853 -0.1853 -0.0138 -0.1853 -0.1853 ...
## $ w8 : num -0.197 -0.197 -0.132 -0.197 -0.197 ...
## $ w9 : num -0.158 -0.158 -0.117 -0.158 -0.158 ...
## $ w10: num -0.51425 -0.026 0.01468 -0.00751 -0.01491 ...
## $ w11: num -0.155 -0.155 -0.155 -0.155 -0.155 ...
## $ w12: num -0.308 0.424 0.44 -0.308 -0.308 ...
## $ y : Factor w/ 2 levels "1","2": 2 2 2 2 2 2 2 2 2 2 ...
```

Data Splitting

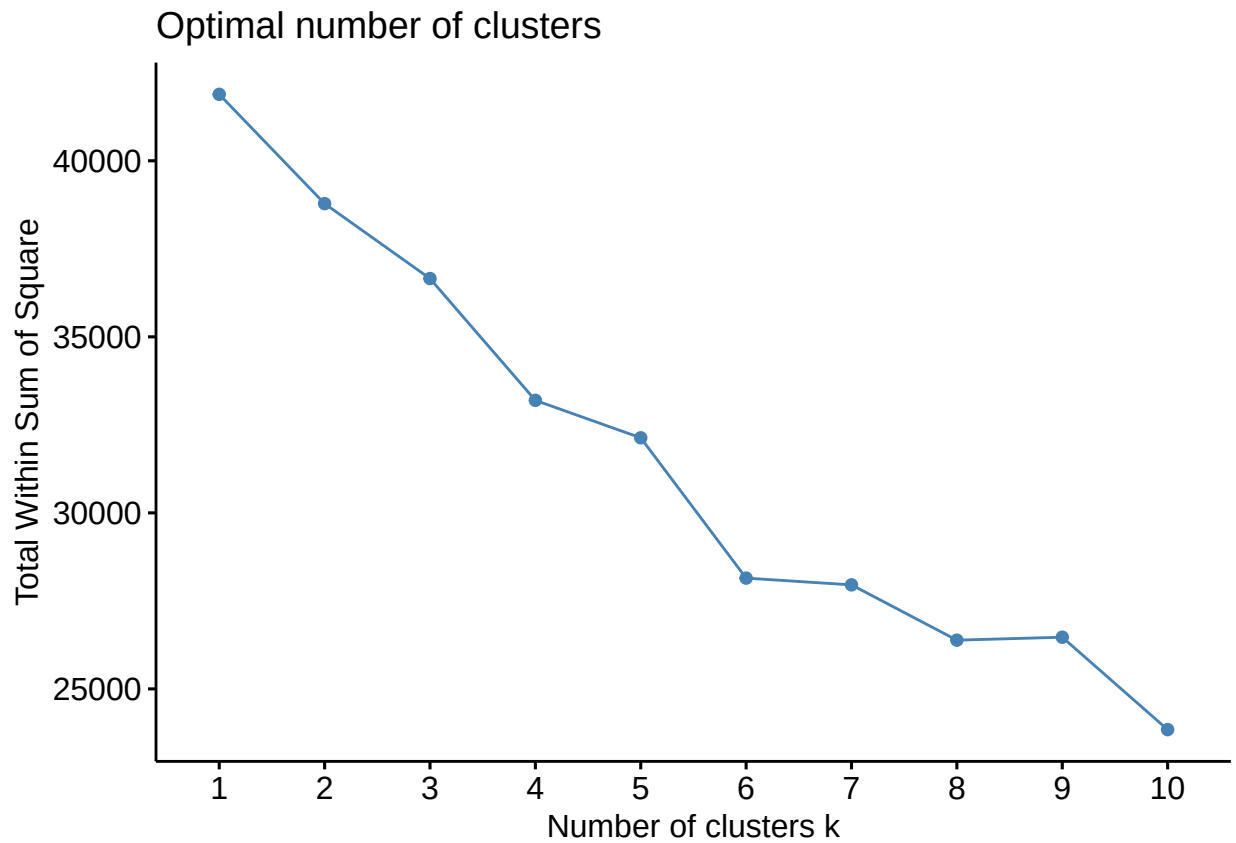
```
set.seed(123)
training.samples <- data$y %>% createDataPartition(p = 0.75, list = FALSE)
train.data <- data[training.samples, ]
test.data <- data[-training.samples, ]
```

CA

K-means with k=2

```
train.data_km <- subset(train.data, select=-c(y))
km_2 <- kmeans(train.data_km, centers = 2, nstart = 25)
```

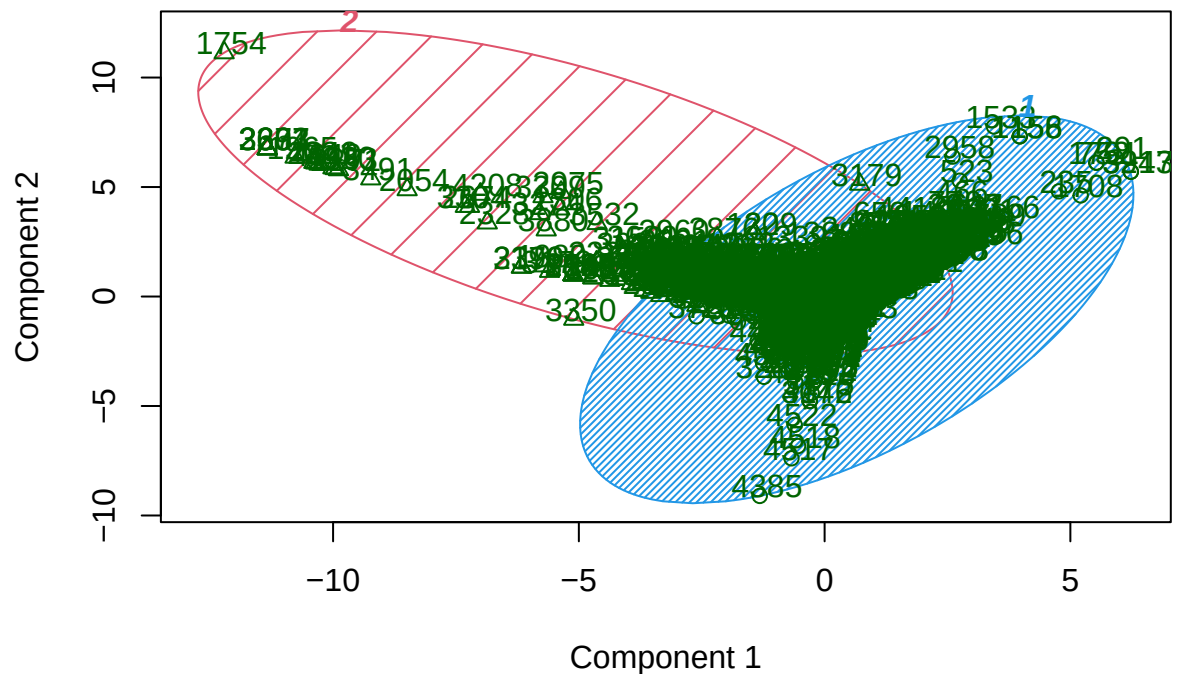
```
fviz_nbclust(train.data_km, kmeans, method = "wss")
```



Scree plot

```
clusplot(  
  train.data_km,  
  km_2$cluster,  
  main = "K-means Clustering",  
  color = TRUE,  
  shade = TRUE,  
  labels = 2,  
  lines = 0  
)
```

K-means Clustering



2D Cluster

These two components explain 23.96 % of the point variability.

```
table(
  factor(km_2$cluster),
  train.data$y
)
```

Confusion Matrix

```
##
##      1    2
##  1 1934 1355
##  2  157    5
```

```
clusters <- factor(
  km_2$cluster,
  levels = c(1,2)
)

cm_k <- confusionMatrix(
  clusters,
  train.data$y
)
cm_k
```

Confusion Matrix and Statistics

```
##
##           Reference
## Prediction    1    2
##           1 1934 1355
##           2  157   5
##
##           Accuracy : 0.5619
##           95% CI : (0.5451, 0.5785)
##       No Information Rate : 0.6059
##       P-Value [Acc > NIR] : 1
##
##           Kappa : -0.0844
##
## Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.924916
##           Specificity : 0.003676
##       Pos Pred Value : 0.588021
##       Neg Pred Value : 0.030864
##           Prevalence : 0.605911
##       Detection Rate : 0.560417
##       Detection Prevalence : 0.953057
##       Balanced Accuracy : 0.464296
##
##       'Positive' Class : 1
##
```

```
cm_k$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity
## 0.9249163
```

```
cm_k$byClass["Specificity"]
```

Specificity

```
## Specificity
## 0.003676471
```

```
cm_k$overall["Accuracy"]
```

Overall Accuracy

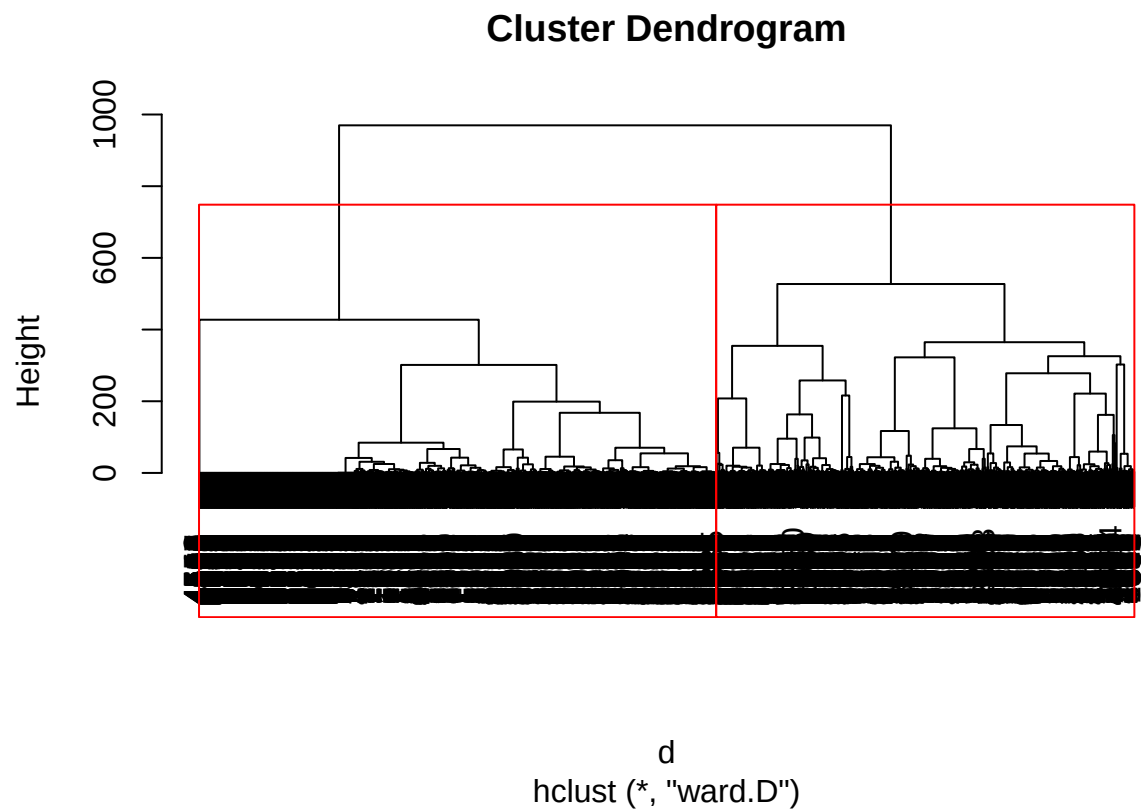
```
## Accuracy
## 0.5618661
```

Hierarchical Clustering

```
d <- dist(  
  train.data,  
  method = "euclidean"  
)
```

Distance Matrix

```
H.fit <- hclust(  
  d,  
  method = "ward.D"  
)  
  
plot(H.fit)  
  
rect.hclust(  
  H.fit,  
  k=2,  
  border = "red"  
)
```



Ward.D


```
##      1    2
##  1 1359  732
##  2   549  811
```

```
clusters <- factor(
  groups,
  levels =c(1,2)
)

cm_h <- confusionMatrix(
  train.data$y,
  clusters
)
cm_h
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    1    2
##              1 1359  732
##              2   549  811
##
##              Accuracy : 0.6288
##              95% CI : (0.6124, 0.645)
##              No Information Rate : 0.5529
##              P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.2406
##
##  Mcnemar's Test P-Value : 3.675e-07
##
##              Sensitivity : 0.7123
##              Specificity : 0.5256
##              Pos Pred Value : 0.6499
##              Neg Pred Value : 0.5963
##              Prevalence : 0.5529
##              Detection Rate : 0.3938
##              Detection Prevalence : 0.6059
##              Balanced Accuracy : 0.6189
##
##              'Positive' Class : 1
##
```

```
cm_h$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity
##  0.7122642
```

```
cm_h$byClass["Specificity"]
```

Specificity

```
## Specificity  
## 0.5255995
```

```
cm_h$overall["Accuracy"]
```

Overall Accuracy

```
## Accuracy  
## 0.6288032
```

Result

In terms of confusion matrix, Hierarchical Clustering performs better.

K means has very low specificity. So overall is Hierarchical Clustering performs better

PCA

Data Pre-processing

```
train.data_pca <- subset(train.data,select=-c(y))
```

remove y from training

Compute PC

```
data.pca <- prcomp(  
  train.data_pca ,  
  center = TRUE,  
  scale.=TRUE  
)
```

```
summary(data.pca)
```

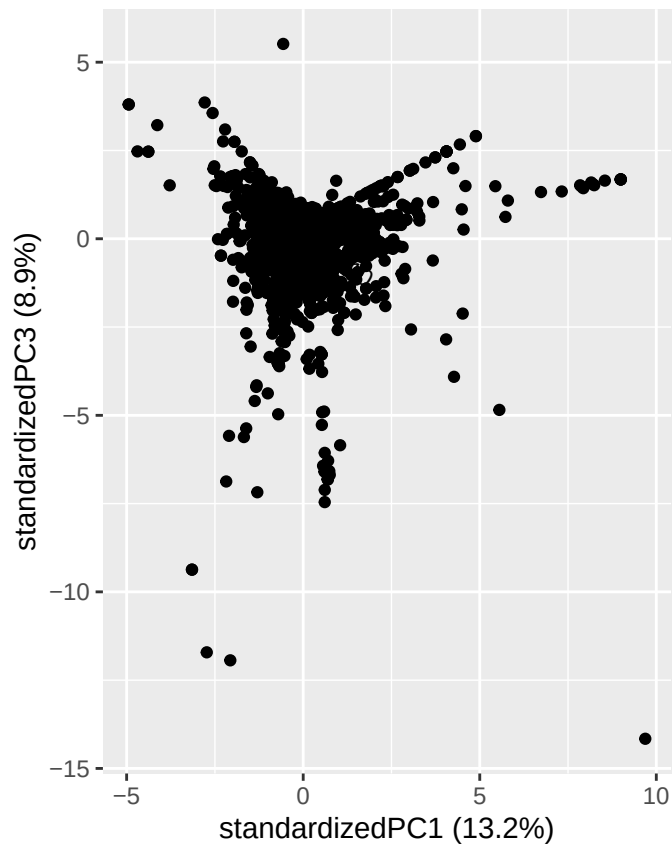
Summary

```
## Importance of components:
##           PC1    PC2    PC3    PC4    PC5    PC6    PC7
## Standard deviation  1.2607 1.1337 1.03498 1.00871 0.99573 0.98392 0.97504
## Proportion of Variance 0.1325 0.1071 0.08927 0.08479 0.08262 0.08068 0.07922
## Cumulative Proportion 0.1325 0.2396 0.32882 0.41362 0.49624 0.57692 0.65614
##           PC8    PC9    PC10    PC11    PC12
## Standard deviation  0.96179 0.95237 0.92908 0.89740 0.79105
## Proportion of Variance 0.07709 0.07558 0.07193 0.06711 0.05215
## Cumulative Proportion 0.73323 0.80881 0.88074 0.94785 1.00000
```

How many PCs are need to explain at least 80% of the variance We need 9 PCs to explain at least 80% of the variance

Biplot

```
ggbiplot(data.pca, choices = c(1,3))
```



SVM

Default Model Fitting

```
set.seed(123)
model <- train(
  y ~., data = train.data, method = "svmLinear",
  trControl = trainControl("cv", number = 10),
)
```

```
predicted.classes <- model %>% predict(test.data)
```

Prediction

```
cm_svm <- confusionMatrix(
  factor(predicted.classes),
  factor(test.data$y)
)
cm_svm
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1    2
##           1 665 113
##           2  32 340
##
##           Accuracy : 0.8739
##           95% CI : (0.8533, 0.8926)
##           No Information Rate : 0.6061
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7274
##
##  Mcnemar's Test P-Value : 3.06e-11
##
##           Sensitivity : 0.9541
##           Specificity : 0.7506
##           Pos Pred Value : 0.8548
##           Neg Pred Value : 0.9140
##           Prevalence : 0.6061
##           Detection Rate : 0.5783
##           Detection Prevalence : 0.6765
##           Balanced Accuracy : 0.8523
##
##           'Positive' Class : 1
##
```

```
cm_svm$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity  
## 0.954089
```

```
cm_svm$byClass["Specificity"]
```

Specificity

```
## Specificity  
## 0.7505519
```

```
cm_svm$overall["Accuracy"]
```

Overall Accuracy

```
## Accuracy  
## 0.873913
```

With `tuneGrid = expand.grid(C = seq(0.001, 3, length = 30))`

```
set.seed(123)  
model <- train(  
  y ~., data = train.data, method = "svmLinear",  
  trControl = trainControl("cv", number = 10),  
  tuneGrid = expand.grid(C = seq(0.001, 3, length = 30))  
)
```

```
model$bestTune
```

Best Tune

```
## C  
## 25 2.482931
```

The Optimal C is 2.482931

```

set.seed(123)
model <- train(
  y ~., data = train.data, method = "svmLinear",
  trControl = trainControl("cv", number = 10),
  tuneGrid = expand.grid(C = model$bestTune)
)

predicted.classes <- model %>% predict(test.data)

cm_svm <- confusionMatrix(factor(predicted.classes), factor(test.data$y), positive = '1')

cm_svm

```

Model with best Tune

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1    2
##           1 665 110
##           2  32 343
##
##           Accuracy : 0.8765
##           95% CI : (0.8561, 0.895)
##       No Information Rate : 0.6061
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7334
##
##  Mcnemar's Test P-Value : 1.035e-10
##
##           Sensitivity : 0.9541
##           Specificity : 0.7572
##           Pos Pred Value : 0.8581
##           Neg Pred Value : 0.9147
##           Prevalence : 0.6061
##           Detection Rate : 0.5783
##       Detection Prevalence : 0.6739
##           Balanced Accuracy : 0.8556
##
##           'Positive' Class : 1
##

```

```
cm_svm$byClass["Sensitivity"]
```

Sensitivity

```

## Sensitivity
##    0.954089

```

```
cm_svm$byClass["Specificity"]
```

Specificity

```
## Specificity  
## 0.7571744
```

```
cm_svm$overall["Accuracy"]
```

Overall Accuracy

```
## Accuracy  
## 0.8765217
```

With TuneLength = 10, radial basis kernel

```
set.seed(123)  
model <- train(  
  y ~ .,  
  data = train.data,  
  method = "svmRadial",  
  trControl = trainControl("cv", number = 10),  
  tuneLength = 10  
)
```

```
model$bestTune
```

Best Tune

```
##          sigma C  
## 4 0.4725552 2
```

The Optimal sigma and C are 0.4725552 and 2

```
predicted.classes <- model %>% predict(test.data)
```

Prediction

```
cm_svm <- confusionMatrix(
  factor(predicted.classes),
  factor(test.data$y)
)
cm_svm
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1    2
##           1 643  68
##           2  54 385
##
##           Accuracy : 0.8939
##           95% CI : (0.8747, 0.9111)
##           No Information Rate : 0.6061
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.7766
##
##  Mcnemar's Test P-Value : 0.2392
##
##           Sensitivity : 0.9225
##           Specificity : 0.8499
##           Pos Pred Value : 0.9044
##           Neg Pred Value : 0.8770
##           Prevalence : 0.6061
##           Detection Rate : 0.5591
##           Detection Prevalence : 0.6183
##           Balanced Accuracy : 0.8862
##
##           'Positive' Class : 1
##
```

```
cm_svm$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity
##  0.9225251
```

```
cm_svm$byClass["Specificity"]
```

Specificity


```
## Specificity
## 0.8498896
```

```
cm_svm$overall["Accuracy"]
```

Overall Accuracy

```
## Accuracy
## 0.893913
```

TuneLength = 3, polynomial basis kernel

```
set.seed(123)
model <- train(
  y ~.,
  data = train.data,
  method = "svmPoly",
  trControl = trainControl("cv", number = 10),
  tuneLength = 3
)
```

```
model$bestTune
```

Best Tune

```
## degree scale C
## 27 3 0.1 1
```

```
predicted.classes <- model %>% predict(test.data)
```

Prediction

```
cm_svm<-confusionMatrix(
  factor(predicted.classes),
  factor(test.data$y),
  positive = '1'
)
cm_svm
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1    2
##           1 651  80
##           2  46 373
##
##           Accuracy : 0.8904
##           95% CI : (0.8709, 0.9079)
##           No Information Rate : 0.6061
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7675
##
## Mcnemar's Test P-Value : 0.003283
##
##           Sensitivity : 0.9340
##           Specificity : 0.8234
##           Pos Pred Value : 0.8906
##           Neg Pred Value : 0.8902
##           Prevalence : 0.6061
##           Detection Rate : 0.5661
##           Detection Prevalence : 0.6357
##           Balanced Accuracy : 0.8787
##
##           'Positive' Class : 1
##
```

```
cm_svm$byClass["Sensitivity"]
```

Sensitivity

```
## Sensitivity
## 0.9340029
```

```
cm_svm$byClass["Specificity"]
```

Specificity

```
## Specificity
## 0.8233996
```

```
cm_svm$overall["Accuracy"]
```

Overall Accuracy

```
## Accuracy
## 0.8904348
```

Result

The best Sensitivity

SVM with linear

0.954089

The best Specificity

SVM with RBF, TuneLength = 10

0.8498896

The best Overall Accuracy

SVM with RBF, TuneLength = 10

0.893913

Overall the best model is SVM with RBF because according to the sensitivity, Specificity, overall accuracy, although sensitivity is not the highest but relatively speaking SVM with RBF has overall highest result.